

Prerana Educational and Social Trust®
PES INSTITUTE OF TECHNOLOGY
AND MANAGEMENT

Approved by AICTE, New Delhi | Affiliated to VTU, Belagavi | Recognized by Govt. of Karnataka
NBA Accredited Programs CSE, CV, ECE, ISE, ME & EEE | Accredited by NAAC: A Grade



DEPARTMENT OF
INFORMATION SCIENCE AND ENGINEERING

PROJECT MANAGEMENT (WITH GIT)
Code: 1BCSL307A
Semester: III

Name of the Student	
USN	
Academic Year	



NH-206, Sagar Road, Shivamogga-577204



Prerana Educational and Social Trust ®
PES INSTITUTE OF TECHNOLOGY & MANAGEMENT

NH-206, SAGAR ROAD, SHIVAMOGGA-577204



(Approved by AICTE, New Delhi | Affiliated to VTU, Belagavi | Recognized by Govt. of Karnataka)

NBA Accredited Programs: CSE, CV, ECE, ISE, EEE & ME

Telephone:- +91 9380741865, 8147053063 | Website:- pestrust.edu.in/pesitm | Email:- principal_pesitm@pes.edu

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

ACADEMIC YEAR 2025-26



PROJECT MANAGEMENT (WITH GIT)

Code: 1BCSL307A

Semester: III

Prepared by

Mrs. Thara K L

Assistant Professor, ISE

Date: 07/09/2026

Approved by

Dr. Mouneshachari S.

Professor and HoD, ISE



INSTITUTE VISION

To be the most preferred institution for engineering & management education, research & Entrepreneurship by creating professionally superior & ethically strong global manpower.

INSTITUTE MISSION

- *Provide high-quality engineering and management education with technical excellence for global requirements.*
- *Promote research and entrepreneurship with collaboration, modern facilities, and strong mentorship.*
- *Build ethically grounded graduates who succeed globally, lead with integrity and commit to lifelong learning and social responsibility.*

DEPARTMENT VISION

To create technically competent, ethically strong and skilled IT Professionals

DEPARTMENT MISSION

- M1:** *Enhancing the knowledge through ongoing research and professional development program by incorporating effective teaching and learning methods.*
- M2:** *To prepare students to be successful in their professional career and continuous learners by offering a strong technical foundation with professional and ethical responsibilities.*

PROGRAM EDUCATION OBJECTIVES:

- PEO1:** *Graduates will adapt to the emerging technologies in the field of Information Science to build professionally successful careers.*
- PEO2:** *Graduates will be adapt to provide the solution for the problems in the field of Information Technology and fulfill the societal requirements.*
- PEO3:** *Graduates will function effectively in IT field by creative thinking, continuous learning and having good ethics.*

PROGRAM SPECIFIC OUTCOMES:

- PSO1:** *Analyze the logics, Algorithms and Information systems to enable the students to design and implement the software.*
- PSO2:** *Evaluate working of Hardware and Software aspects of computer using appropriate tools.*
- PSO3:** *Apply the standard practices in software development to deliver quality products.*

COURSE OUTCOMES

COs	Description
At the end of the course the student will be able to:	
CO1	Demonstrate the use of basic Git repository commands for version control operations
CO2	Implement branch management strategies for collaborative software development
CO3	Apply collaboration techniques using remote repositories and distributed version control systems.
CO4	Configure Git tags, releases, and advanced Git operations for effective project management.
CO5	Modify Git history to maintain repository integrity and support project requirements.

PROGRAM OUTCOMES	
Our graduates will be able to	
PO1.	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions to complex engineering problems.
PO2.	Problem Analysis: Identify, formulate, review research literature and analyse complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3.	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4.	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions. (WK8)
PO5.	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling, recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6.	The Engineer and the World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for their impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5 and WK7)
PO7.	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws. (WK9)
PO8.	Individual and Collaborative Team Work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9.	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10.	<i>Project Management and Finance:</i> <i>Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.</i>
PO11.	<i>Life-Long Learning:</i> <i>Recognize the need for, and have the preparation and ability for</i> <i>i) independent and life-long learning, ii) adaptability to new and emerging technologies, and iii) critical thinking in the broadest context of technological change. (WK8)</i>

General Instructions to Students (Laboratory Specific)

- Students must enter the laboratory **only during the allotted lab hours** and must carry their **lab manual/record book** and required stationery.
- Wearing a **college ID card** and maintaining **proper dress code** is mandatory in the laboratory.
- Students should strictly follow the **instructions given by the lab instructor** and perform experiments only after understanding the procedure.
- **Food, beverages, and mobile phones** are strictly prohibited inside the laboratory.
- Handle all **equipment, systems, and instruments carefully**. Any damage due to negligence will be viewed seriously.
- Students must **not change system settings, install software, or connect external devices** without permission from the lab instructor.
- Before starting the experiment, ensure that the **system/equipment is in proper working condition** and report any issues immediately.
- Maintain **silence and discipline** in the laboratory to ensure a good learning environment.
- All experiments must be **completed within the scheduled lab hours**, and outputs/results should be shown to the instructor for evaluation.
- Students should **record observations and results neatly** in the lab record and submit them within the stipulated time.
- At the end of the lab session, students must **log out of systems, shut down equipment properly**, and keep their workstation clean.
- **Unauthorized copying of programs or results** is strictly prohibited and will be treated as malpractice.
- Students must ensure **data security and privacy** while working on systems provided in the laboratory.
- In case of emergency or equipment malfunction, **inform the lab instructor immediately** and follow safety instructions.
- Students are responsible for maintaining the **cleanliness and proper arrangement** of the laboratory.

Safety Instructions to Students (Laboratory Specific)

- Students must follow **all safety guidelines** and instructions given by the lab instructor at all times.
- Entry into the laboratory is permitted **only under the supervision of the instructor/lab assistant**.
- Students should be aware of the **location and proper use of safety equipment** such as fire extinguishers, first-aid kits, and emergency exits.
- **Running, shouting, or any form of indiscipline** is strictly prohibited inside the laboratory.
- Electrical equipment must be handled with **dry hands only** to avoid electric shock.
- Do not **switch on/off equipment or power supplies** without the permission of the lab instructor.
- Any **faulty equipment, loose wires, or abnormal behavior** of systems must be reported immediately.
- **Cables, connectors, and instruments** should be arranged properly to avoid tripping hazards.
- Students must **not attempt to repair or modify equipment** on their own.
- In case of **fire, electric shock, or any emergency**, students should remain calm and follow the instructor's directions.
- **Sharp tools, hot surfaces, or moving parts** (where applicable) must be handled with extreme care.
- Avoid **overloading electrical sockets** and do not use unauthorized extension cords.
- Maintain a **clean and dry working area** to prevent accidents.
- Immediately report **any injury or accident**, even if minor, to the lab instructor.
- Students must **evacuate the laboratory in an orderly manner** when an emergency alarm is raised.

SYLLABUS

Project Management (with Git)			
Course Code	1BCSL307A	Scheme	2025
Type of Course	AEC (Laboratory)	Semester	3
Teaching Hours/Week (L:T: P)	0:0:2	CIE Marks	50
Total Hours of Pedagogy L:T:P:SL:TW	0:0:28:2:30	SEE Marks	50
Credits	1	Total Marks	100
Type of Examination	Practical	Exam Hours	03
Course outcome (Course Skill Set)			
At the end of the course, the student will be able to:			
1. Demonstrate the use of basic Git repository commands for version control operations. 2. Implement branch management strategies for collaborative software development 3. Apply collaboration techniques using remote repositories and distributed version control systems. 4. Configure Git tags, releases, and advanced Git operations for effective project management. 5. Modify Git history to maintain repository integrity and support project requirements.			
SET OF EXPERIMENTS			
1. Setting Up and Basic Commands: Initialize a new Git repository in a directory. Create a new file and add it to the staging area, and commit the changes with an appropriate commit message. 2. Creating and Managing Branches: Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master." 3. Creating and Managing Branches: Write the commands to stash your changes, switch branches, and then apply the stashed changes 4. Collaboration and Remote Repositories: Clone a remote Git repository to your local machine. 5. Collaboration and Remote Repositories: Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch. 6. Collaboration and Remote Repositories: Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge. 7. Git Tags and Releases: Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository. 8. Advanced Git Operations: Write the command to cherry-pick a range of commits from "source-branch" to the current branch. 9. Analysing and Changing Git History: Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message? 10. Analysing and Changing Git History: Write the Git command to display all commits made by the author "JohnDoe" between "January 1, 2025" and "December 31, 2025." 11. Analysing and Changing Git History: Write the command to display the last five commits in the repository's history. 12. Analysing and Changing Git History: Write the command to undo the changes introduced by the commit with the ID "abc123".			

Evaluation Rubrics

Assessment Structure:

The assessment for each course is equally divided between Continuous Internal Evaluation (CIE) and the Semester End Examination (SEE), with each component carrying **50% weightage** (i.e., 50 marks each).

The CIE marks awarded shall be based on the continuous evaluation of the laboratory report using a defined set of rubrics. Each experiment report can be evaluated for 30 marks. Total marks for the all report shall be scaled down to 30 marks. The laboratory test (duration 03 hours) at the end of the last week of the semester /after completion of all the experiments (whichever is early) shall be conducted for 50 marks and Marks scored shall be scaled down to 20 marks. Final average of marks for report and test shall be out of 50 marks.

List of Experiments

Exp. No.	Experiment	Page No.
1	<i>Setting Up and Basic Commands: Initialize a new Git repository in a directory. Create a new file and add it to the staging area, and commit the changes with an appropriate commit message.</i>	
2	<i>Creating and Managing Branches: Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "featurebranch" into "master."</i>	
3	<i>Creating and Managing Branches: Write the commands to stash your changes, switch branches, and then apply the stashed changes.</i>	
4	<i>Collaboration and Remote Repositories: Clone a remote Git repository to your local machine.</i>	
5	<i>Collaboration and Remote Repositories: Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.</i>	
6	<i>Collaboration and Remote Repositories: Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.</i>	
7	<i>Git Tags and Releases: Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.</i>	
8	<i>Advanced Git Operations: Write the command to cherry-pick a range of commits from "source-branch" to the current branch.</i>	
9	<i>Analysing and Changing Git History: Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message?</i>	
10	<i>Analysing and Changing Git History: Write the Git command to display all commits made by the author "JohnDoe" between "January 1, 2025" and "December 31, 2025".</i>	
11	<i>Analysing and Changing Git History: Write the command to display the last five commits in the repository's history.</i>	
12	<i>Analysing and Changing Git History: Write the command to undo the changes introduced by the commit with the ID "abc123".</i>	

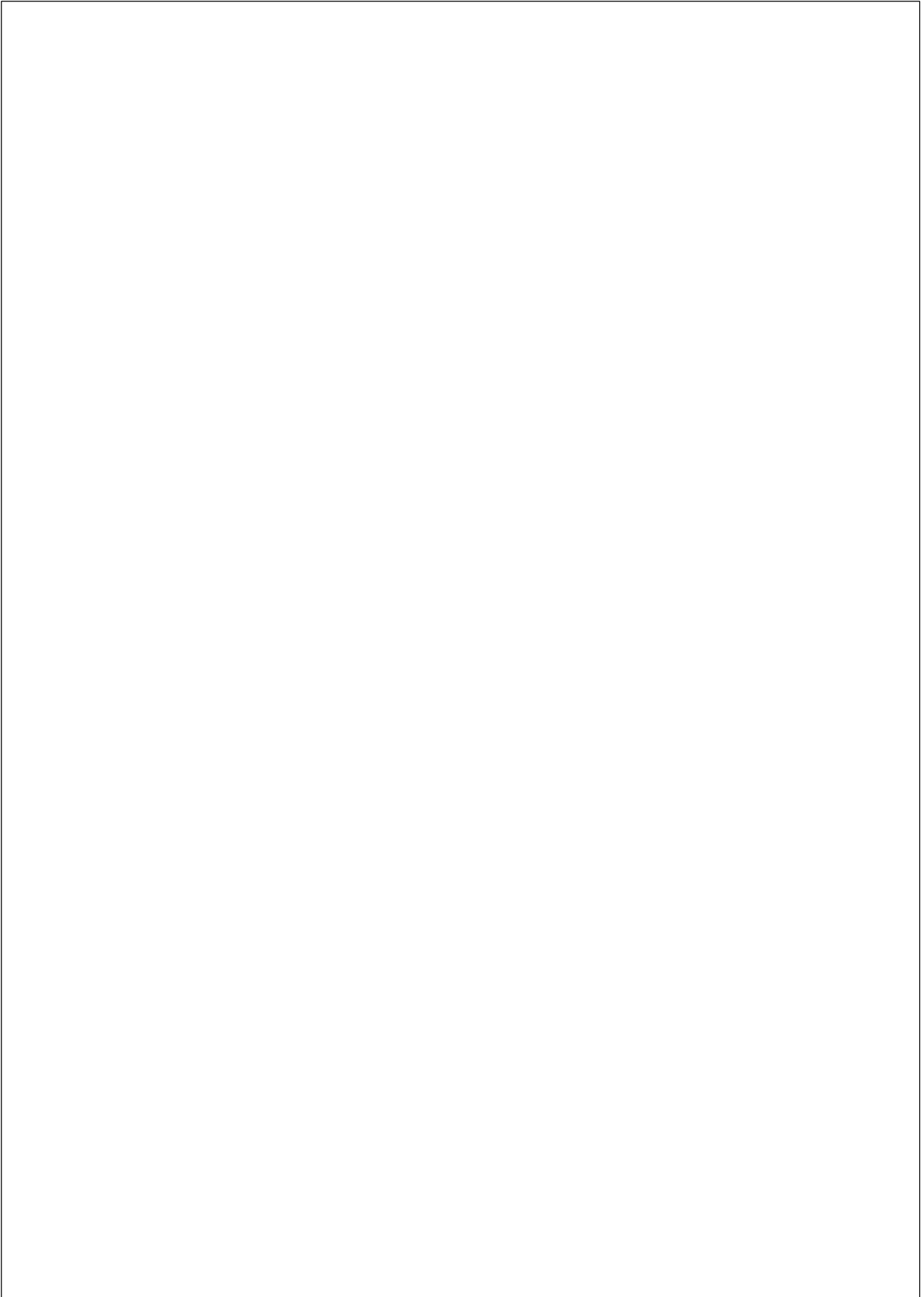
Experiment 1

Setting Up and Basic Commands: Initialize a new Git repository in a directory. Create a new file and add it to the staging area, and commit the changes with an appropriate commit message.

Aim: To initialize a Git repository, create and stage a new file, and commit the changes to the repository with an appropriate commit message.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

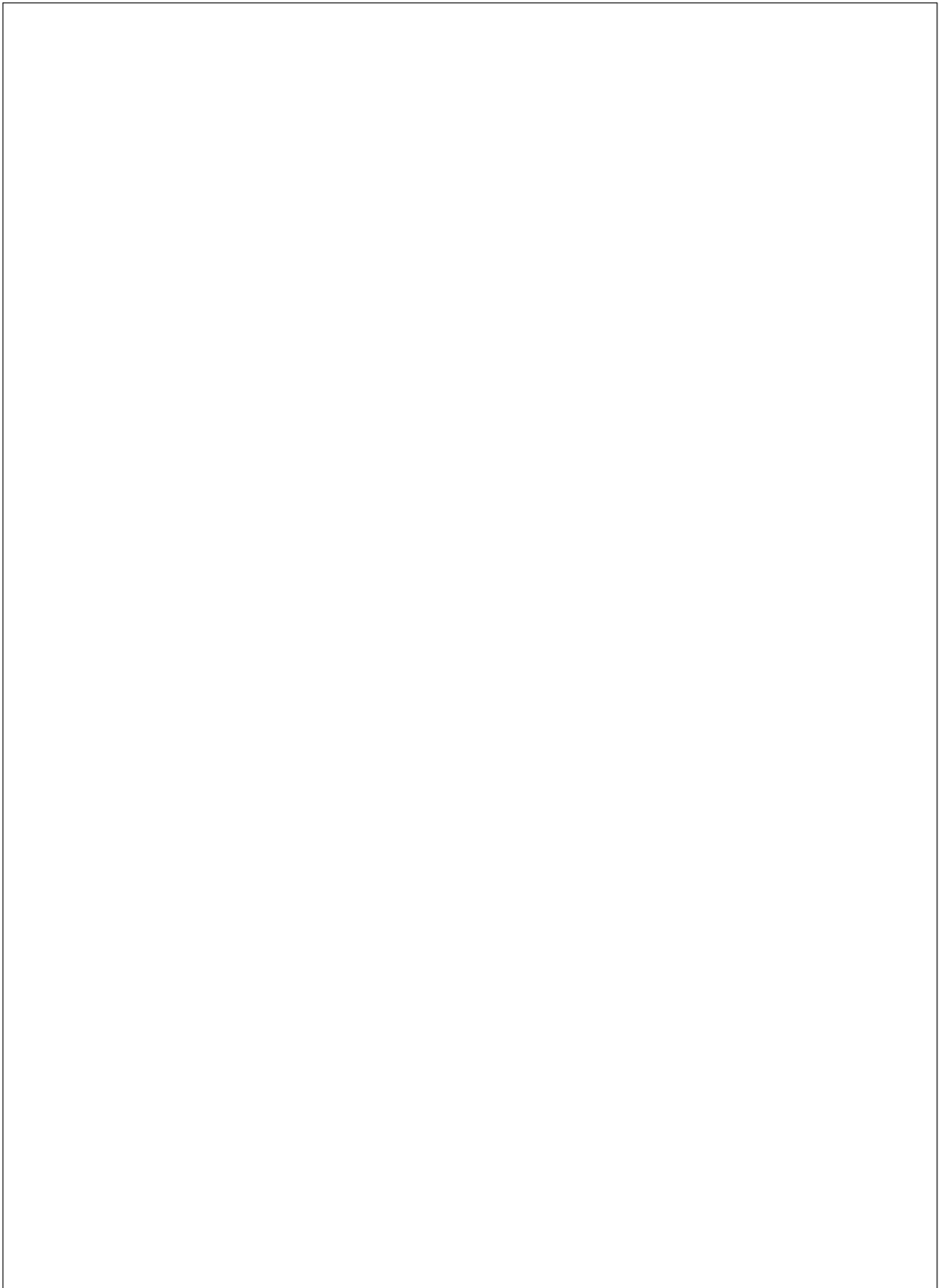
Experiment 2

. Creating and Managing Branches: Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "featurebranch" into "master."

Aim: To understand branch creation, branch switching, and merging in Git using a feature branch and the master branch.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

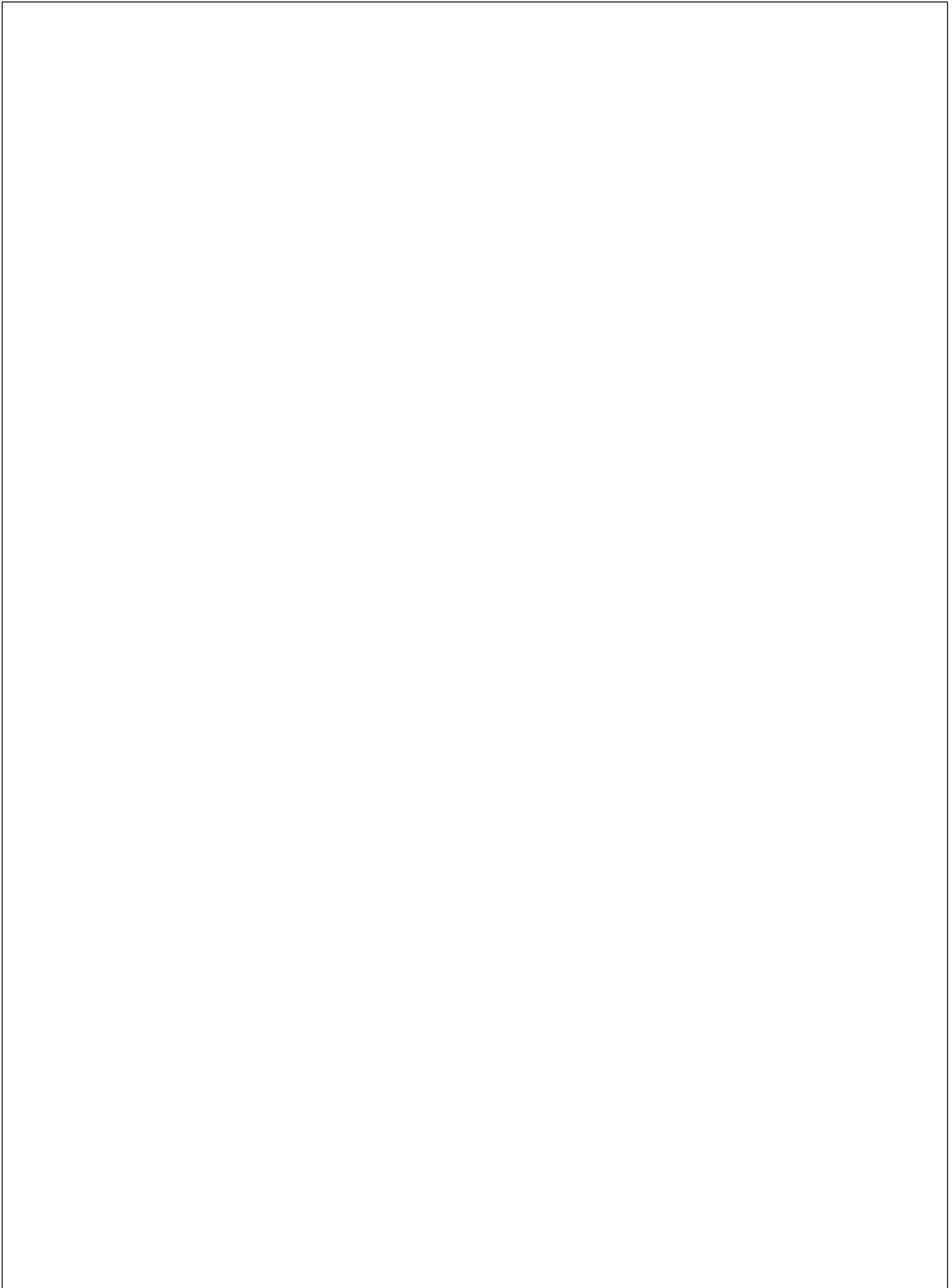
Experiment 3

Creating and Managing Branches: Write the commands to stash your changes, switch branches, and then apply the stashed changes

Aim: To understand Git stash operations by temporarily saving uncommitted changes, switching to another branch, and applying the stashed changes when required.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

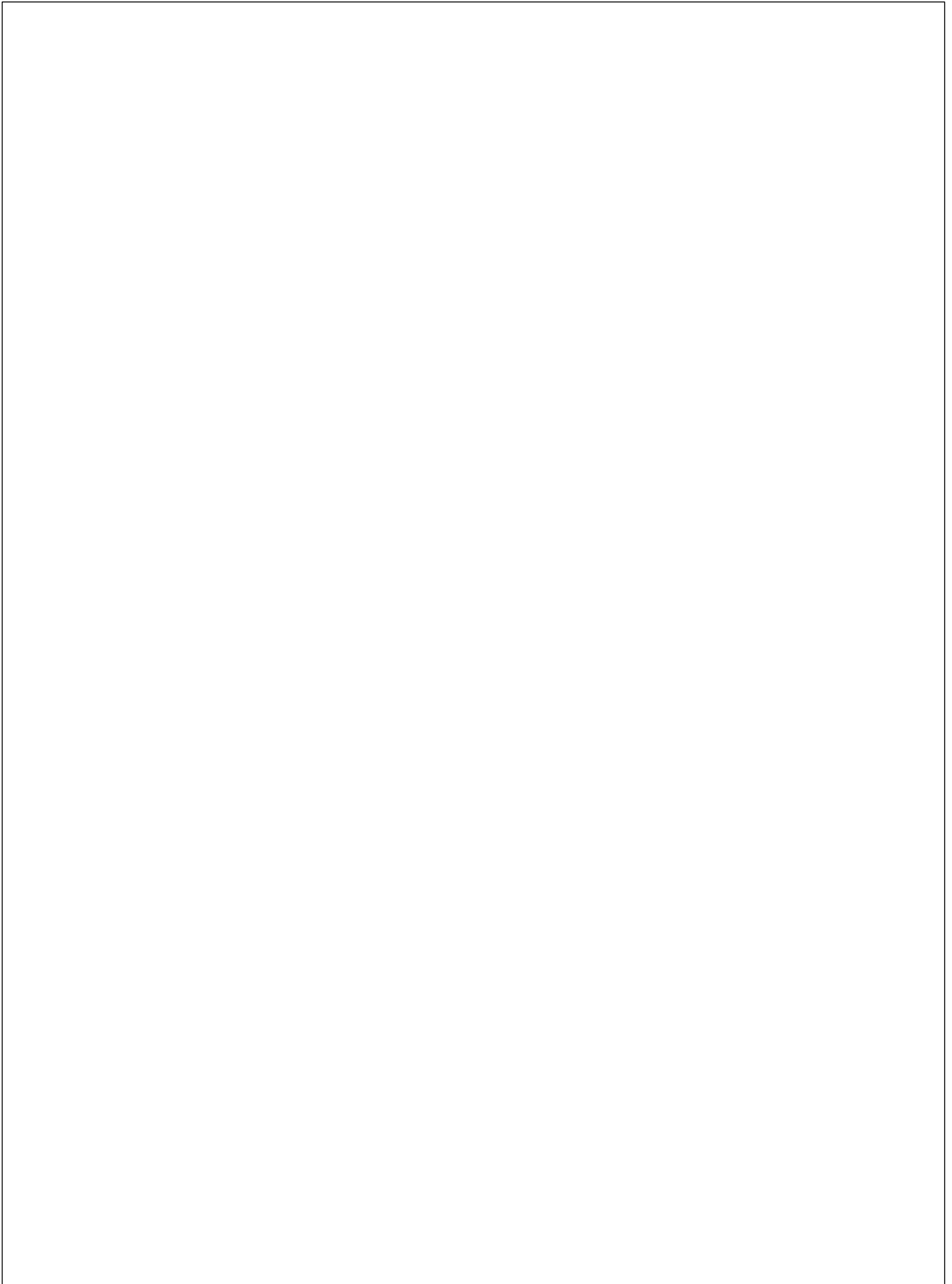
Experiment 4

Collaboration and Remote Repositories: Clone a remote Git repository to your local machine.

Aim: To clone a remote Git repository to the local machine and understand how to work with a copy of a repository hosted on a remote server.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

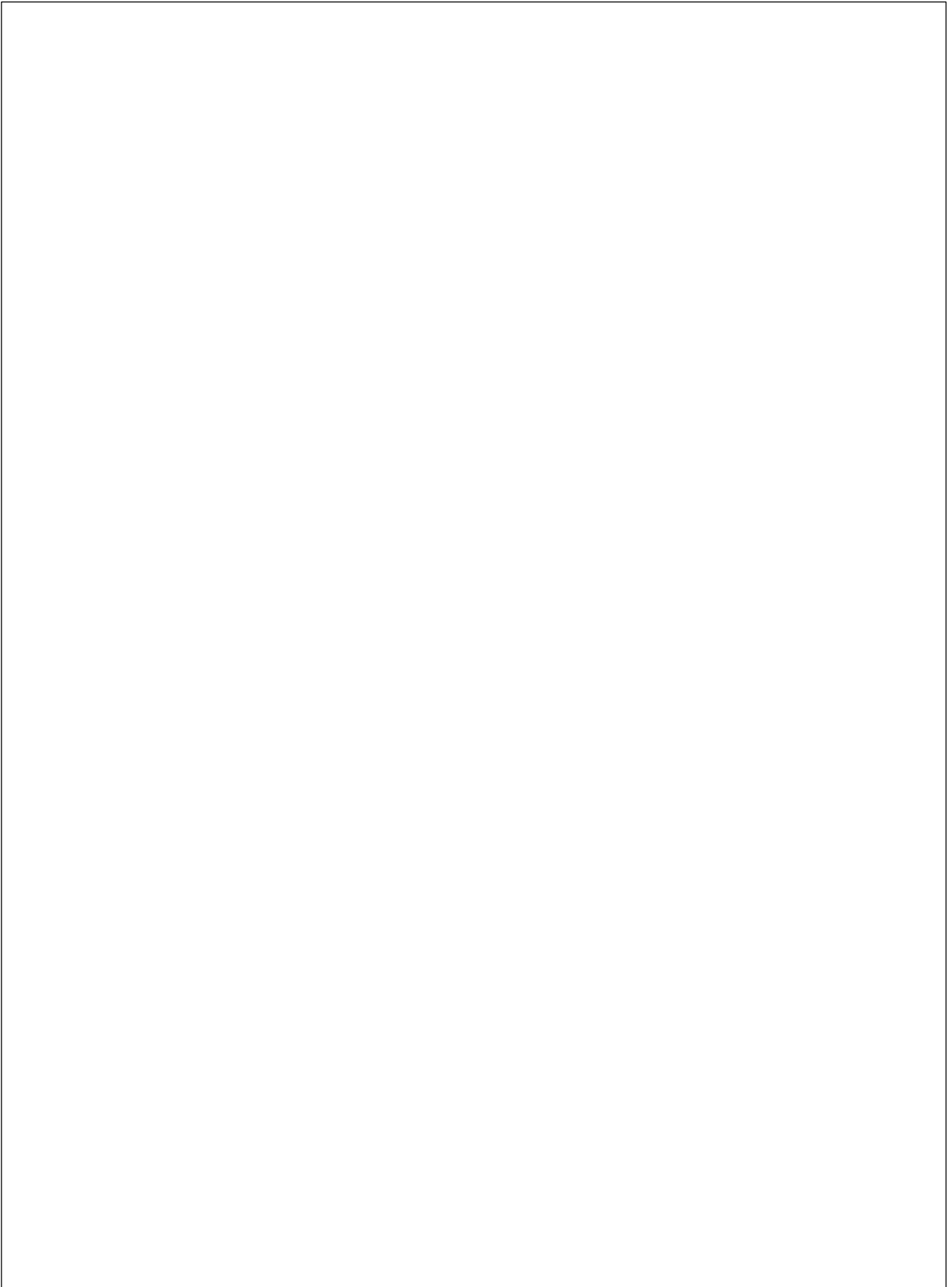
Experiment 5

Collaboration and Remote Repositories: Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.

Aim: To understand how to fetch remote changes and rebase the local branch with the latest updates from the remote repository.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

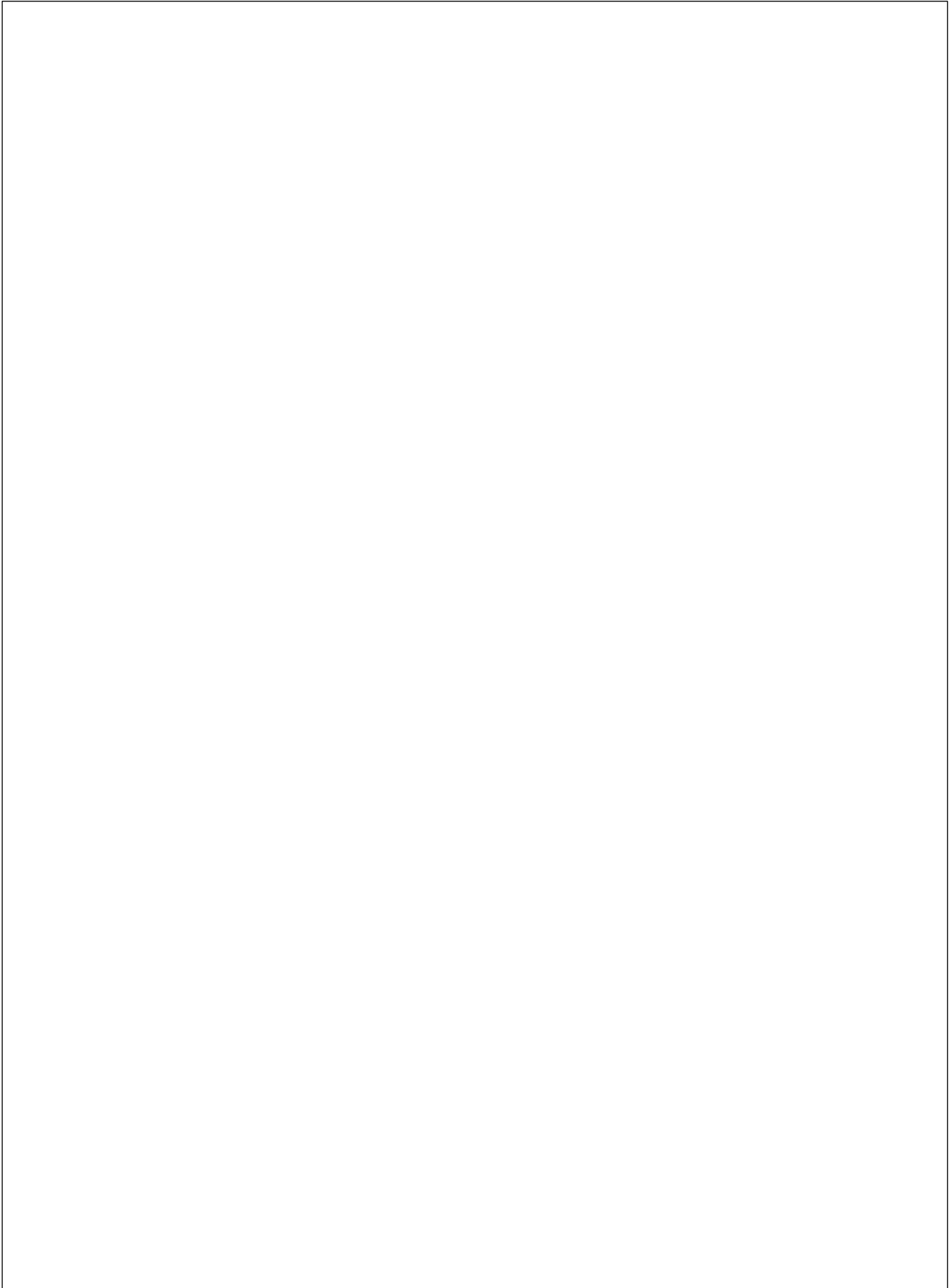
Experiment 6

Collaboration and Remote Repositories: Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.

Aim: To merge the `feature-branch` into the `master` branch and provide a custom commit message for the merge operation.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

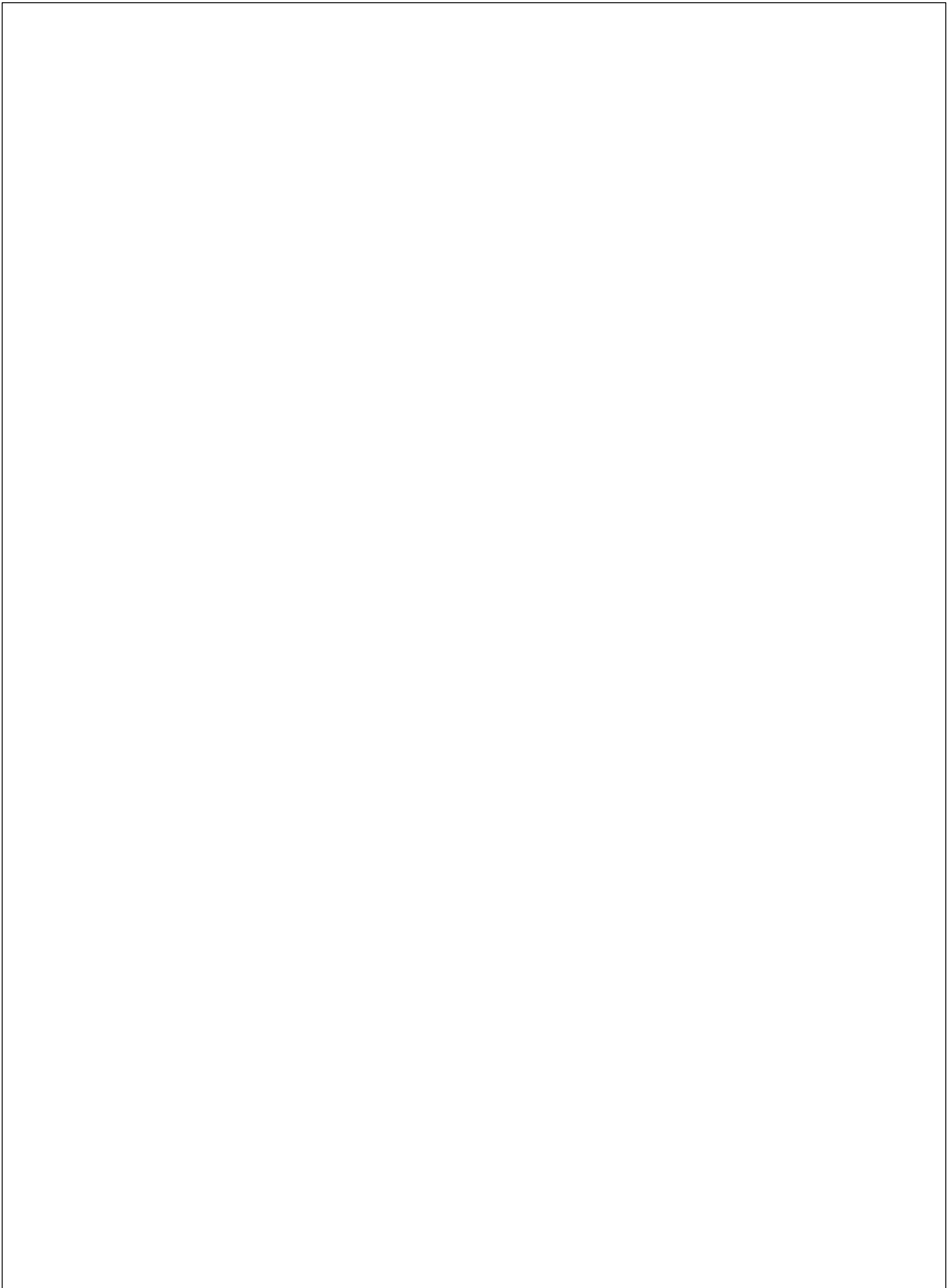
Experiment 7

Git Tags and Releases: Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.

Aim: To understand how to create and manage lightweight tags in Git for marking specific commits or versions(create a lightweight Git tag named v1.0 for a commit in the local repository).

Tools / Soft wares required:

Programs:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

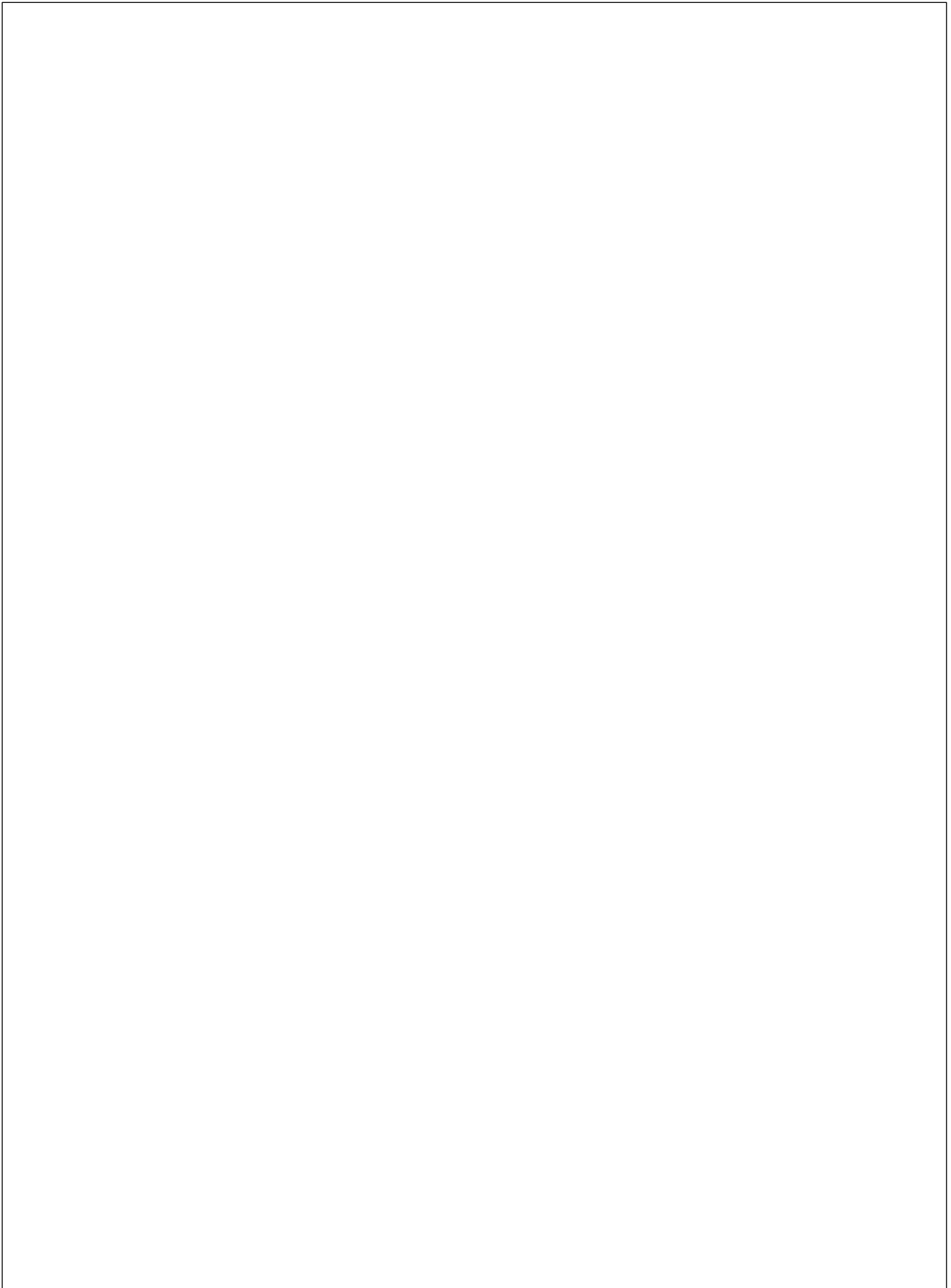
Experiment 8

Advanced Git Operations: Write the command to cherry-pick a range of commits from "source-branch" to the current branch.

Aim: To understand and perform cherry-picking of a range of commits from the `source-branch` to the current branch using Git.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

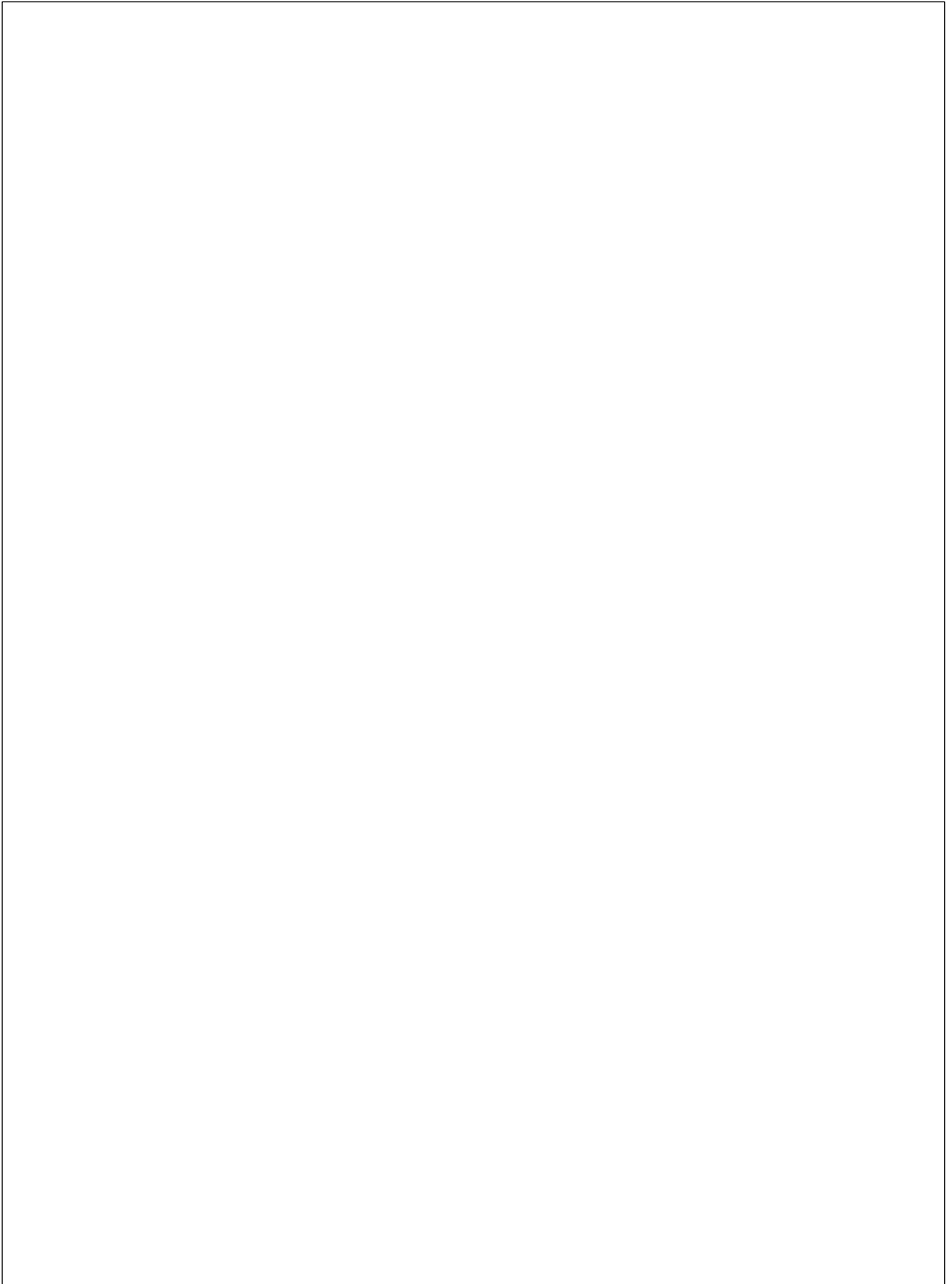
Experiment 9

Analysing and Changing Git History: Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message?

Aim: To analyze a specific Git commit by using its commit ID to view details such as the author, commit date, commit message, and changes made in that commit.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

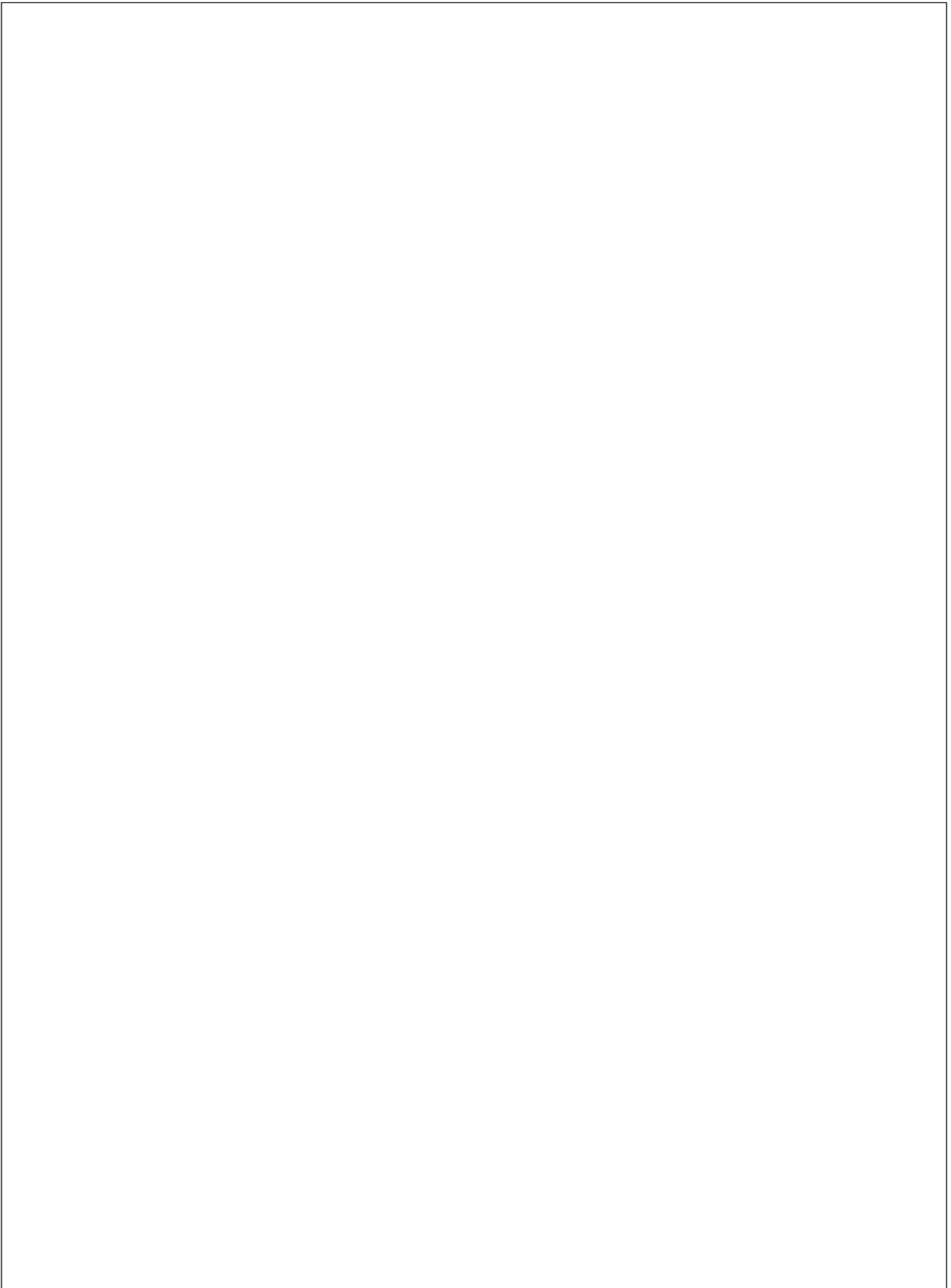
Experiment 10

Analysing and Changing Git History: Write the Git command to display all commits made by the author "JohnDoe" between "January 1, 2025" and "December 31, 2025".

Aim: To use Git log commands to display all commits made by a specific author, JohnDoe, within a specified date range.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

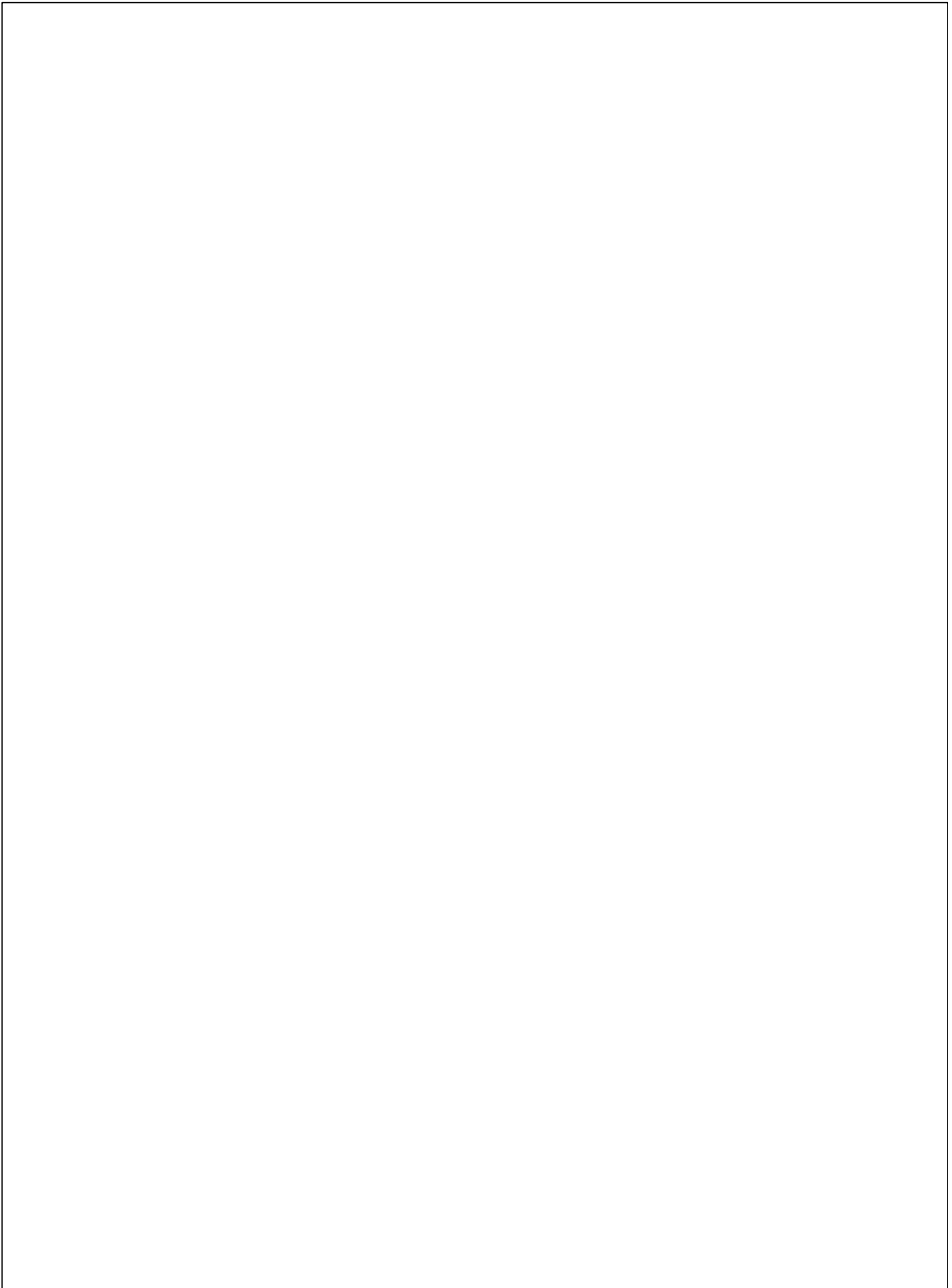
Experiment 11

Analysing and Changing Git History: Write the command to display the last five commits in the repository's history

Aim: To display the last five commits in the Git repository's history using the Git log command. This helps in reviewing the recent changes and understanding the sequence of commits made in the repository.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

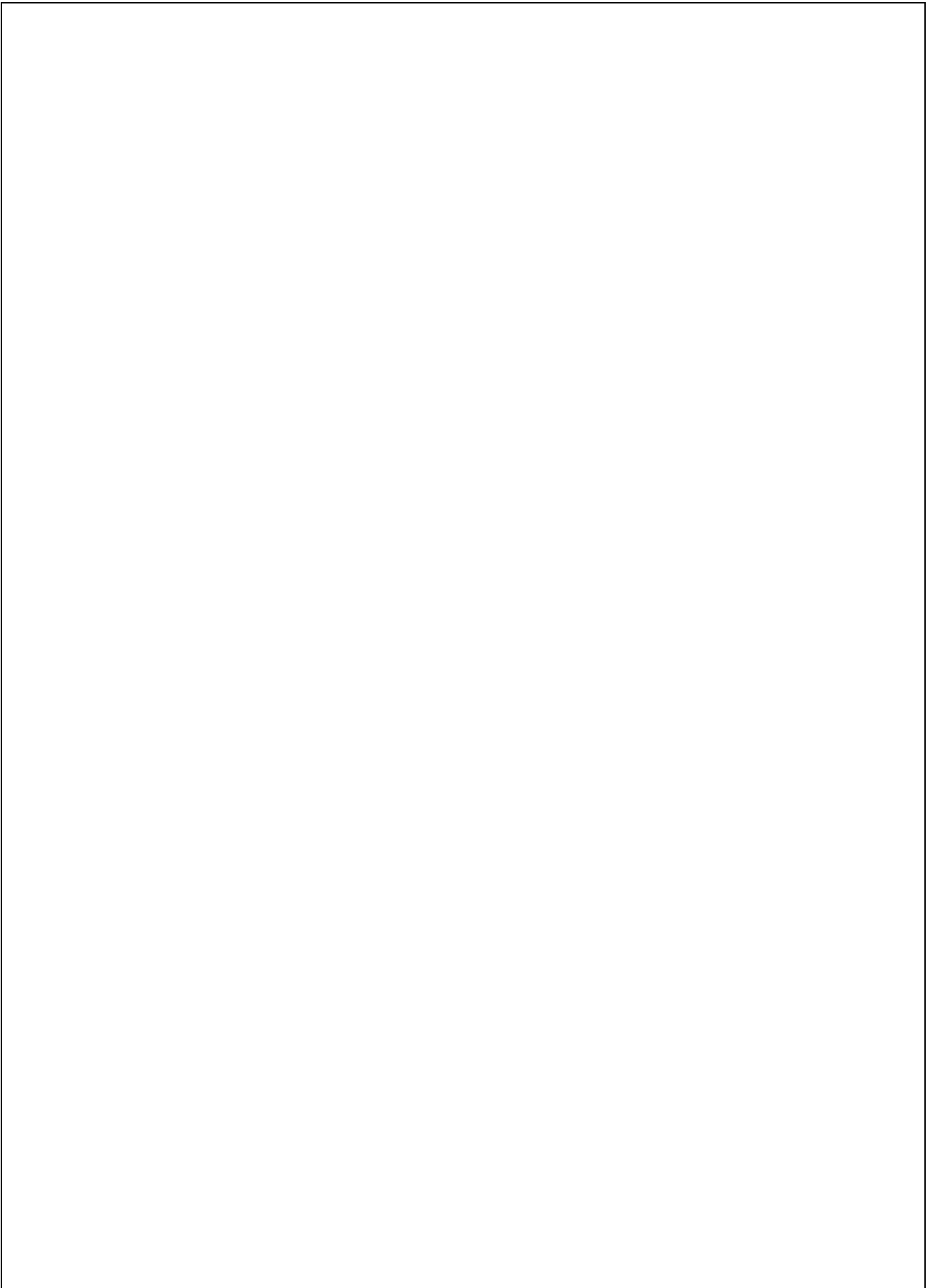
Experiment 12

Analysing and Changing Git History: Write the command to undo the changes introduced by the commit with the ID "abc123".

Aim: To undo the changes introduced by a specific commit using its commit ID with the Git revert command. This helps in safely reversing the effect of a particular commit while preserving the existing commit history.

Tools / Soft wares required:

Program:



Observations & Calculations:

Date of Conduction:		Signature of Student:	
Particulars		Max. Marks	Marks Secured
Conduction	Preparation / Awareness	05	
	Execution of Experiment	05	
Viva-voce		05	
TOTAL		15	
Signature of Faculty with Date			

Viva Questions

Q1. What is Git?

Git is a distributed version control system used to track changes in source code.

Q2. What is GitLab?

GitLab is a web-based platform that hosts Git repositories and provides collaboration features.

Q3. Difference between Git and GitLab?

- Git: Version control tool.
- GitLab: Cloud platform that hosts Git repositories.

Q4. What is a Git repository?

A Git repository is a storage location that contains project files and their version history.

Q5. What is a Git branch?

A branch is a separate line of development used to work on changes independently.

Q6. How do you create a branch named feature-branch?

`git branch feature-branch`

Q7. How do you switch to the master branch?

`git checkout master`

Q8. How do you merge feature-branch into master?

`git checkout master`

`git merge feature-branch`

Q9. What is merging in Git?

Merging combines changes from one branch into another branch.

Q10. Why are branches used in Git?

Branches allow developers to work on different features without directly affecting the main code.

Q11. What is Git stash?

Git stash temporarily saves uncommitted changes so that we can switch branches safely.

Q12. How do you stash your current changes?

`git stash`

Q13. How do you apply the stashed changes?

`git stash apply`

Q14. What is the difference between git stash apply and git stash pop?

Apply keeps the changes in the stash, while pop applies the changes and removes them from the stash.

Q15. What is a remote repository?

A remote repository is a Git repository stored on a server such as GitLab or GitHub.

Q16. What is cloning in Git?

Cloning creates a local copy of a remote repository.

Q17. What command is used to clone a repository?

`git clone <repository-url>`

Q18. What is git fetch used for?

`git fetch` downloads the latest changes from a remote repository without automatically merging them.

Q19. What is git rebase?

Rebase reapplies local commits on top of another branch to maintain a cleaner history.

Q20. How do you fetch and rebase onto the updated remote master branch?

`git fetch origin`

git rebase origin/master

Tags and Cherry-Pick

Q21. What is a Git tag?

A tag is a name used to identify a specific commit, usually for a version or release.

Q22. How do you create a lightweight tag named v1.0?

git tag v1.0

Q23. What is cherry-picking in Git?

Cherry-picking applies the changes of a selected commit to the current branch.

Q24. How do you cherry-pick a commit?

git cherry-pick <commit-id>

Q25. How do you cherry-pick a range of commits?

git cherry-pick <start-commit>^..<end-commit>

Git History

Q26. How do you view the details of a specific commit?

git show <commit-id>

Q27. How do you display commits made by a particular author during 2025?

git log --author="JohnDoe" --since="2025-01-01" --until="2025-12-31"

Q28. How do you display the last five commits?

git log -5

Q29. What is git revert?

git revert creates a new commit that reverses the changes made by an earlier commit.

Q30. How do you undo the changes introduced by commit abc123?

git revert abc123

Additional Experiments:

Experiment 1: Basic Git Workflow

Create a repository, create a file, check its status, stage the file, and commit the changes.

Experiment 2: Handling Merge Conflicts

Create two branches, modify the same file differently, merge the branches, identify the conflict, and resolve it.

Experiment 3: Push and Pull with GitLab

Create a local repository, connect it to a GitLab remote repository, push changes, and pull changes from the remote repository.

Experiment 4: Working with .gitignore

Create a .gitignore file and configure Git to ignore selected files and folders.

Experiment 5: Git Diff and Undo Operations

Compare file changes using `git diff` and demonstrate `git restore`, `git restore --staged`, and `git revert`.